



cpi'fp

Los Enlaces

DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

ARQUITECTURA MVC

1. PATRONES DE SOFTWARE

Los **patrones de software** son *soluciones comprobadas a problemas comunes en el desarrollo de software*. La arquitectura MVC, de hecho, es un patrón, porque se ha probado infinidad de veces y se adapta a la perfección a multitud de problemas diferentes. Un patrón:

- Debe haber sido comprobado en otros sistemas.
- Debe ser fácilmente reutilizable.
- Debe ser aplicable a diferentes circunstancias.
- Debe estar bien documentado

1. PATRONES DE SOFTWARE

Tipos de patrones:

- De arquitectura
- De diseño
- De creación de objetos
- De estructura de clases
- De comportamiento
- De dialectos
- De interacción o interfaz de usuario
- De análisis
- De dominio

1. PATRONES DE SOFTWARE

Antes de centrarnos en el patrón MVC, vamos a ver, **solo a modo de ejemplo**, otro tipo de patrón: el patrón **Singleton**, que está considerado un *patrón de diseño*.

Algunos recursos son de tal naturaleza que sólo puede existir una instancia de ese tipo de recurso. Por ejemplo, la conexión a la base de datos a través de un manejador de base de datos. A veces interesa compartir un manejador de base de datos para que el resto de recursos no tengan que conectarse y desconectarse continuamente de la BD, y sólo debería existir una instancia de ese manejador.

1. PATRONES DE SOFTWARE

El patrón Singleton cubre esta necesidad. Un objeto es “singleton” si la aplicación puede generar una y sólo una instancia del mismo.

https://drive.google.com/file/d/1wNIs2PNCa4aVwRZuYSVSegMiVZXamtzT/view?usp=drive_link

Teniendo esta clase, el objeto Singleton puede usarse de este modo:

```
// Las dos variables contendrán, en realidad, el mismo objeto Singleton
$single1 = Singleton::getInstance();
$single2 = Singleton::getInstance();
```



2. PATRONES DE ARQUITECTURA

Existe un problema cuando hablamos de “arquitectura de una aplicación” en el ámbito de las aplicaciones web.

El problema es este: usamos el término “*arquitectura de una aplicación*” para referirnos a dos cosas distintas: la ***arquitectura física*** y la ***arquitectura lógica***.

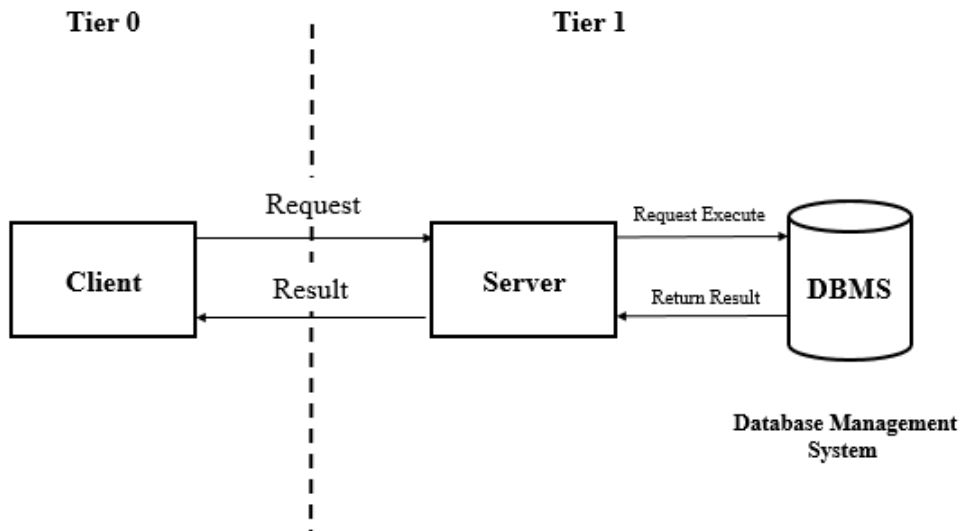
2. PATRONES DE ARQUITECTURA

Una **arquitectura física** de varios niveles (multinivel o *multitier*, en inglés) consiste en un conjunto de ordenadores conectados a una red que ejecutan de forma conjunta una aplicación.

El ejemplo más sencillo es la arquitectura **cliente-servidor**, la más popular en aplicaciones web sencillas: una máquina cliente y una máquina servidor ejecutan alternativamente fragmentos del código, proporcionando al usuario final la sensación de una aplicación unificada.

2. PATRONES DE ARQUITECTURA

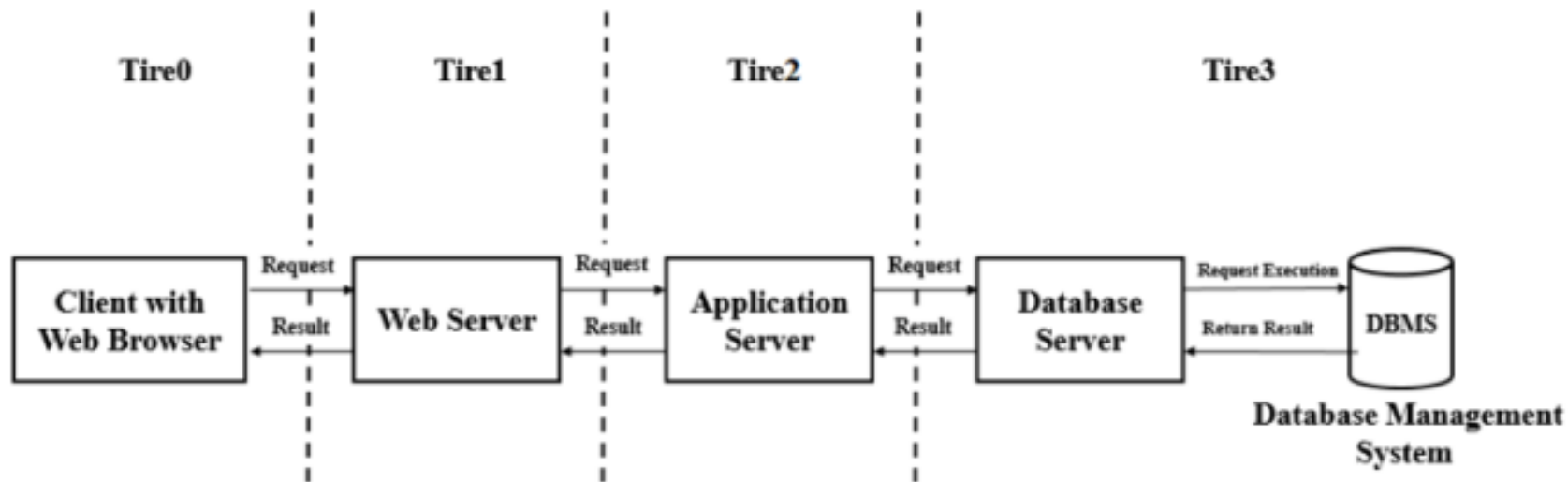
Arquitectura cliente-servidor





2. PATRONES DE ARQUITECTURA

Arquitectura multicapa





2. PATRONES DE ARQUITECTURA

La arquitectura física es algo que incumbe sobre todo a los administradores de sistemas, que son los encargados de montarla, configurarla y mantenerla. Para el programador, sin embargo, suele resultar transparente.

Por ejemplo, cuando nos conectamos a un servidor de bases de datos desde nuestra aplicación, poco importa que ese servidor esté en la misma máquina física o en otra máquina diferente: la forma de conectarse y operar con la base de datos es exactamente la misma.



3. ARQUITECTURA LÓGICA

La arquitectura de una aplicación también puede referirse a sus **capas lógicas** (*layers* en inglés). Es decir, a las capas de software que nosotros, los desarrolladores/as, creamos.

Dividir una aplicación en capas que colaboran entre sí por medio de interfaces bien definidos no es una idea nueva, ni pertenece exclusivamente al ámbito de la programación web. Pero la mayor parte de las aplicaciones web hacen uso de este mecanismo de abstracción.

3. ARQUITECTURA LÓGICA

La idea es fragmentar nuestra aplicación en capas de niveles de abstracción cada vez mayor. En un extremo, la **capa más abstracta** es la que interacciona con el **usuario**: ahí se implementará nuestro interfaz de usuario, o lo que en aplicaciones web se llama *front-end*.

En el otro extremo, la **capa menos abstracta** es la que está en contacto con el **hardware** de la máquina. Bueno, en el caso de una aplicación web, no hay contacto directo con el hardware, sino con otras capas como el sistema operativo, el servidor web o el gestor de bases de datos.

3. ARQUITECTURA LÓGICA

Ventajas de las arquitecturas multicapa:

- Desarrollar en paralelo cada capa (mayor rapidez de desarrollo).
- Aplicaciones más robustas gracias al encapsulamiento. Cada capa se implementa en una clase, y cada clase hace su trabajo sin preocuparse por cómo funcionan las otras internamente.
- El mantenimiento es más sencillo.
- Más flexibilidad para añadir módulos.
- Más seguridad, al poder aislar (relativamente) cada capa del resto.

3. ARQUITECTURA LÓGICA

Ventajas de las arquitecturas multicapa:

- Mejor escalabilidad: es más fácil hacer crecer al sistema.
- Mejor rendimiento (aunque esto podría discutirse: puedes hacer un sistema multicapa con un rendimiento desastroso y un sistema monolítico que vaya como un tiro. Pero, en general, es más fácil mejorar el rendimiento trabajando en cada capa por separado).
- Es más fácil hacer el control de calidad, incluyendo la fase de pruebas.



3. ARQUITECTURA LÓGICA

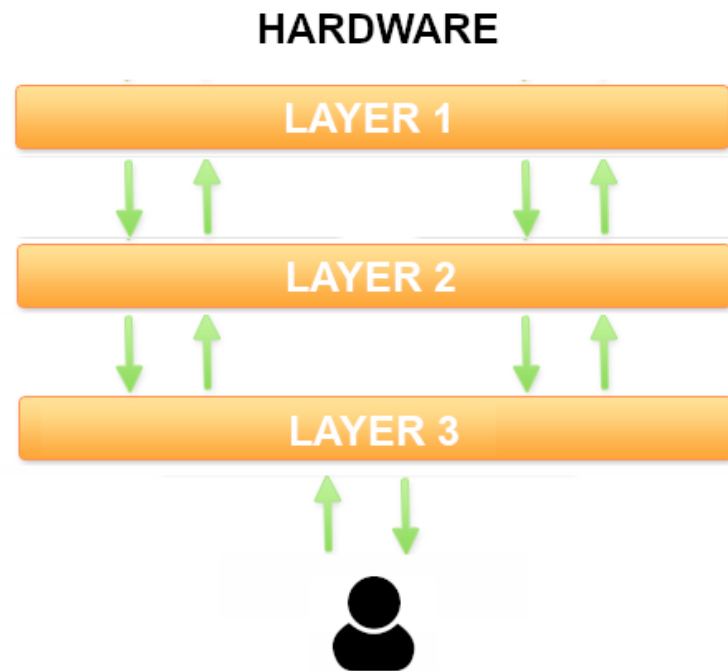
Inconvenientes de las arquitecturas multicapa:

- El único inconveniente reseñable de las arquitecturas multicapa es que pueden hacer que una aplicación muy simple se vuelva artificialmente más compleja de lo necesario. Pero eso solo sucede en aplicaciones muy, pero que muy simples. Para cualquier aplicación convencional, la arquitectura multicapa simplifica el diseño en lugar de complicarlo.

4. ARQUITECTURA MVC

MVC es tan solo una arquitectura multicapa estandarizada. Una arquitectura de 3 capas, para ser exactos.

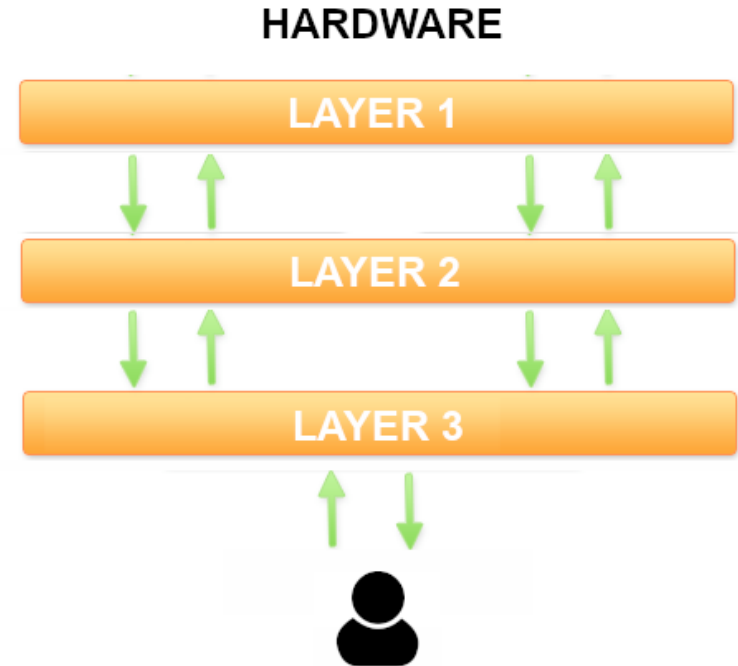
Este es el esquema de una arquitectura de 3 capas:





4. ARQUITECTURA MVC

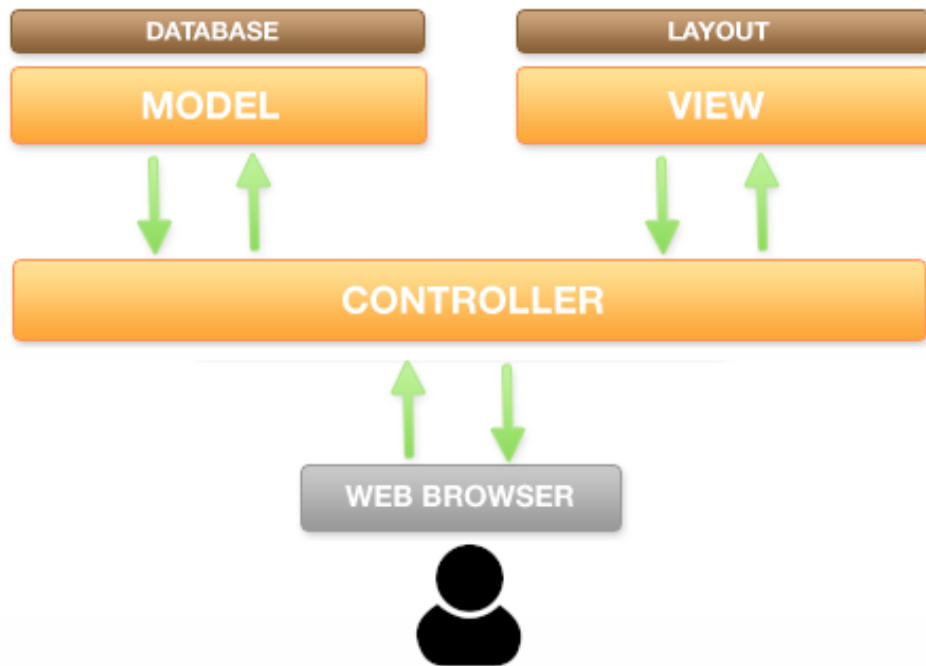
Cada capa ejecuta una parte de la solución, y entre ellas colaboran para formar la aplicación completa. La capa inferior interactúa con el usuario; la capa superior con la máquina (donde dice “hardware”, debería decir “cualquier cosa menos abstracta que nuestro programa”).



4. ARQUITECTURA MVC

Si a esas tres capas les ponemos nombres (*modelo*, *vista* y *controlador*) y modificamos un poco el esquema, ya lo tenemos: la **arquitectura MVC**.

Es decir, la arquitectura MVC solo es un caso particular de la arquitectura en 3 capas.



4. ARQUITECTURA MVC

Todo esto está muy bien como construcción teórica, pero ¿cómo afecta a la hora de programar? ¿Qué clases tienes que crear? ¿Qué parte del código hay que poner en cada clase?

En la práctica, es más simple de lo que parece. Y lo mejor es que el 99,99% de las aplicaciones web encajan como un guante en esta arquitectura. Es decir, apenas tendremos que hacer trabajo de diseño previo porque, si es una aplicación web, ya sabemos qué clases tendremos que construir: las que nos indique la arquitectura MVC.

5. EJEMPLO INCREMENTAL

Este es un código sencillo y bien comentado, y que se va complicando muy poco a poco, en pasos incrementales, desde un código clásico monolítico hasta una implementación completa de un MVC.

https://drive.google.com/file/d/1CaoQ6ewV411lrzX1v1XCq8LNOYeGxqaG/view?usp=drive_link

El ejemplo con el que vamos a trabajar es este: supongamos que queremos programar una pequeña aplicación web que nos permita hacer publicaciones en una especie de blog simplificado. Esas publicaciones se guardan como registros en una tabla de una base de datos.



5. EJEMPLO INCREMENTAL

```
// Conectamos con la base de datos
$db = new mysqli('mi-host', 'mi-usuario', 'mi-clave', 'mi-base-de-datos');
// Lanzamos una consulta para recuperar los artículos que haya en la base de datos
$res = $db->query('SELECT fecha, titulo FROM articulo');

// Generamos una tabla HTML con el resultado de la consulta
echo "<h1>Listado de Artículos</h1>";
echo "<table>";
echo "<tr> <th>Fecha</th> <th>Titulo</th> </tr>";
while ($row = $res->fetch_array()) {    // Recorremos fila a fila el resultado de la consulta
    echo "<tr> <td> ".$row['fecha']." </td> <td> ".$row['titulo']." </td> </tr>";
}
echo "</table>";
$db->close();    // Cerramos la conexión con la BD
```



5. EJEMPLO INCREMENTAL

Esta solución se denomina monolítica, porque incluye todo el código necesario en el mismo bloque.

Por supuesto, para un ejemplo tan simple como éste el código monolítico es más que suficiente, pero en un sistema más complejo pronto empieza a convertirse en un monstruo inmanejable.



5. EJEMPLO INCREMENTAL

PRIMERA MEJORA: CONTROLADOR + VISTA

Vamos a aproximarnos un poco a la solución MVC separando ese código monolítico en dos bloques (que guardaremos en archivos distintos):

- Un controlador (archivo *index.php*)
- Una vista (archivo *showAllArticles.php*)

https://drive.google.com/drive/folders/1rr3iy8iMCuY2nReles588iVFuS-YyIt2?usp=drive_link



5. EJEMPLO INCREMENTAL

Primero, el **controlador**. Se encargará de recuperar los datos, pero *no de mostrarlos*. Generar el interfaz de usuario, es decir, el HTML, será la labor que le dejaremos a la vista. El controlador preparará esos datos y los empaquetará en un array para que estén disponibles en la vista. Y la vista la insertaremos en el controlador con un *include()*.

https://drive.google.com/file/d/12wm5LxF98C9JWmnFlwcs3hT-WCTsDYEJ/view?usp=drive_link



5. EJEMPLO INCREMENTAL

La **vista** que mostrará los datos del array contiene un código muy semejante al de la solución monolítica, solo que ahora estará ubicada en un archivo aparte (*showAllArticles.php*) y hará un bucle sobre el array de resultados que le ha preparado el controlador.

https://drive.google.com/file/d/1WMe0XtGgickl6C87XGVJcYZ84ND8yqRh/view?usp=drive_link

5. EJEMPLO INCREMENTAL

SEGUNDA MEJORA: MODELO, VISTA Y CONTROLADOR

En esta segunda mejora, dividiremos el código en tres bloques:

- Un **modelo** para los artículos (archivo *articles.php*). Contendrá una clase con un método que se encargará de acceder a la base de datos y empaquetar el resultado de la consulta en un array.
- Una **vista** (archivo *showAllArticles.php*). Se encargará de generar el HTML con el resultado de la consulta.
- Un **controlador** (archivo *index.php*). Se encargará de invocar al modelo y a la vista en el orden correcto.



5. EJEMPLO INCREMENTAL

Por lo tanto, el **controlador** (*index.php*), al extraer de él todo lo que tenga que ver con la base de datos, se queda en algo tan sencillo como esto:

https://drive.google.com/file/d/10IXyG5_1TuDejYrqgCRO_zv-L6-Gl4bR/view?usp=drive_link

```
<?php
    include('articles.php');           // En este archivo estará el modelo
    $articulos = Model::getAll();     // Este método del modelo nos devuelve la
    lista de artículos
    include('showAllArticles.php');    // En este archivo estará la vista
?>
```

5. EJEMPLO INCREMENTAL

El **modelo** (*articles.php*) consta de una clase con solo un método (de momento) encargado de consultar todos los artículos y devolverlos empaquetados en un array:

https://drive.google.com/file/d/1BLaTLr3fp6AWUB-mD6Ke1V9giUzWVKi9/view?usp=drive_link

```
class Articles {  
    public function getAll()  
    {  
        $db = new mysqli('mi-host', 'mi-usuario', 'mi-clave', 'mi-base-de-datos');  
        ...  
    }  
}
```

5. EJEMPLO INCREMENTAL

Por último, la **vista** (*showAllArticles.php*) será exactamente igual que en la versión anterior, un recorrido por el array de artículos para mostrarlos en formato HTML:

https://drive.google.com/file/d/1bE01GL_zXFoG8bZAen-NlrCW9pQldeCX/view?usp=drive_link

```
<h1>Listado de Articulos</h1>
<table>
    <tr> <th>Fecha</th> <th>Titulo</th> </tr>
<?php
    foreach($articles as $article) {...
```

5. EJEMPLO INCREMENTAL

TERCERA MEJORA: CAPA DE ABSTRACCIÓN DE DATOS

Como no sabemos lo que es el miedo, vamos a complicar nuestro patrón modelo-vista-controlador con una **cuarta capa**: la capa de abstracción de datos.

La idea de esta capa adicional es proporcionar un mecanismo de abstracción respecto del gestor de base de datos concreto que estemos utilizando.

https://drive.google.com/drive/folders/1w4Q3sESNpYGQ_FP_LcsLV6l0pjSL9nmr?usp=drive_link

5. EJEMPLO INCREMENTAL

Si te fijas en el **modelo** de la solución anterior, verás que estamos usando una clase (*mysqli*) y unos métodos que solo funcionan con MySQL o MariaDB. Si quisiéramos cambiar el gestor de base de datos (algo relativamente frecuente), tendríamos que revisar *todos* nuestros modelos, y tal vez modificar y volver a probar miles de líneas de código.

Una forma de independizar nuestra aplicación del gestor de base de datos es programar la **capa de abstracción** con dos o tres métodos genéricos (como *consultar()* para lanzar SELECT o *manipular()* para lanzar INSERT, UPDATE o DELETE).

5. EJEMPLO INCREMENTAL

De ese modo, cuando queramos hacer una consulta desde el modelo, no lo haremos con los métodos de MySQL (como *query()*, *fetch_array()* y similares), sino con los nuestros (*consultar()*, *manipular()*, o como los hayamos querido llamar). Si algún día necesitamos cambiar el gestor de base de datos, solo tendremos que reescribir el código de esa capa de abstracción, es decir, un par de decenas de líneas de código frente a varios miles que teníamos que revisar y probar antes.



5. EJEMPLO INCREMENTAL

Por tanto, en esta tercera mejora vamos a dividir el código en 4 bloques:

- Un **controlador** (archivo *index.php*)
- Una **vista** (archivo *view.php*)
- Un **modelo** en dos capas:
 - Capa de **abstracción de datos** (*db.php*)
 - Capa de **acceso a datos**: el modelo de artículos propiamente dicho (*articles.php*)



5. EJEMPLO INCREMENTAL

El código de la **capa de abstracción** (*db.php*) sería algo así:

https://drive.google.com/file/d/1pOHVTauchBKyu-RUXknEMGwlg_JzNiEF/view?usp=drive_link

```
class Db {  
    private $db; // Aquí guardaremos la conexión con la base de datos  
  
    //Abre la conexión con la base de datos  
    function createConnection($server, $username, $pass, $dbname) {  
        $db = new mysqli($servidor, $usuario, $clave, $dbname);  
        ...  
    }  
}
```

5. EJEMPLO INCREMENTAL

El código del **modelo** () va a hacer uso de la capa de abstracción en lugar de usar los métodos de la clase *mysqli* directamente:

https://drive.google.com/file/d/1yQfqUKYfqVSk2nHI6zav_TW5QL-QeA4u/view?usp=drive_link

```
include "db.php";
class Articles {
    public function getAll() {
        $db = new Db(); // Creamos un objeto para usar nuestra capa de abstracción
        // Conectamos con la BD a través de nuestra capa de abstracción
        $db->createConnection('mi-host', 'mi-usuario', 'mi-clave', 'mi-base-de-datos');
        ...
    }
}
```

5. EJEMPLO INCREMENTAL

El **controlador** y la **vista** son exactamente los mismos que en la solución anterior, así que no vamos a escribir el código de nuevo. Esto es lógico: solo hemos modificado la forma en la que trabaja el modelo, pero gracias al encapsulamiento de los objetos, el resto de la aplicación no se ha enterado de ello.

Controlador: [index.php](#)

Vista: [view.php](#)

5. EJEMPLO INCREMENTAL

CUARTA MEJORA: TRANSFORMACIÓN EN CLASES Y OBJETOS REUTILIZABLES.

Ahora vamos a dejar el código bien organizado y a mostrarlo (casi) completo ([archivos](#)).

Lo que haremos es empaquetarlo todo en clases reutilizables. Observa que sigue siendo el mismo código fuente, solo que empaquetado en clases y métodos. Lo único que queda fuera de una clase es la instanciación inicial del objeto controlador.

5. EJEMPLO INCREMENTAL

Fíjate bien en cómo hemos convertido las **vistas** en una clase ([view.php](#)) con un método *show()* que nos servirá para mostrar cualquier vista y reutilizar el mismo *header* y el mismo *footer*. Cada vista se programará en un archivo independiente que deberemos organizar en directorios y subdirectorios. De momento, nuestra aplicación solo tiene una vista llamada [showAllArticles.php](#), pero se podrían visualizar todas las necesarias usando el método *show()*.

5. EJEMPLO INCREMENTAL

El **punto de entrada** a la aplicación ([index.php](#)) lo hemos dejado preparado para poder añadir nuevas funciones al programa con posterioridad, así como varios controladores.

El método que se ejecutará dependerá no solo de la variable “*action*” que se pasa por GET, sino también de otra variable llamada “*controller*”, que también se pasa por GET, y que contendrá el nombre de la clase del controlador.

5. EJEMPLO INCREMENTAL

Así, para invocar, por ejemplo, el método *showAll()* del controlador [ArticlesController](#), la ruta debería ser esta:

`http://mi-servidor/index.php?controller=ArticlesController&action=showAll`

Este [index.php](#) es tan genérico que te servirá para montar cualquier **aplicación MVC** en el futuro.

Las clases [Articles](#) y [Db](#) son las mismas que hemos visto en la mejora anterior.



5. EJEMPLO INCREMENTAL

QUINTA (Y ÚLTIMA) MEJORA: AÑADIENDO UN MODELO GENÉRICO

En todos los modelos nos solemos encontrar una serie de operaciones que se repiten una y otra vez, como:

- Obtener todos los registros de una tabla.
- Obtener un registro de una tabla buscando por id.
- Borrar un registro a partir de su id.
- Insertar un registro.
- Modificar un registro.



5. EJEMPLO INCREMENTAL

Podemos programar un modelo genérico que haga estas cosas *sea cual sea la tabla de la que se trate*. Así no tendremos que escribir una y otra vez las mismas operaciones para cada uno de los modelos: bastará con que nuestros modelos hereden de este modelo genérico, y todas esas operaciones ya estarán disponibles sin escribir ni una línea de código.

Vamos a llamar *Model* a ese modelo genérico.



[archivos](#)

5. EJEMPLO INCREMENTAL

La clase [Model](#) implementa un método *getAll()* genérico que funcionará con cualquier tabla.

Como el modelo [Articles](#) hereda ahora del modelo genérico, *Model*, resulta que ya posee un método llamado *getAll()* que, por lo tanto, no tenemos que programar. A cambio, todo lo que tenemos que hacer es asignar el valor correcto al atributo *\$this->table*, para que el modelo genérico *Model* sepa el nombre de la tabla con la que tiene que trabajar.

5. EJEMPLO INCREMENTAL

Ampliando *Model* con más funciones genéricas, todas ellas se heredarán en *Articles* (y en cualquier otro modelo de la aplicación). Las únicas funciones que tendríamos que escribir en *Articles* serían las específicas de ese modelo, si es que tiene alguna, aunque la mayoría de los modelos no necesitarán ninguna función específica adicional, quedando así su código reducido a la mínima expresión.

Solo faltaría crear una función *insert()* y otra *update()* para tener un CRUD completo en nuestro modelo genérico.

6. RESUMEN

Los modelos, donde se programa la *lógica de negocio*. De esa forma tan rimbombante se refiere la literatura técnica al acceso a los datos con los filtros, algoritmos y restricciones que el sistema imponga.

Dicho en palabras más sencillas, esto significa que en los modelos debemos colocar todo el código de acceso a la base de datos o a cualquier otro recurso del servidor (como las variables de sesión, por ejemplo). Los modelos deben empaquetar esos datos en objetos estándar de PHP (como arrays) y devolverlos al controlador.

6. RESUMEN

Lo más práctico es **crear un modelo para cada tabla maestra** de la base de datos.

Los *frameworks* automatizan los métodos más típicos de cada modelo, como insertar un registro, borrar, actualizar, consultar uno o consultar todos. Ya veremos de qué forma se las ingenian para hacer todo esto con un mínimo de esfuerzo por nuestra parte y, por supuesto, sin escribir el mismo código una y otra vez.

6. RESUMEN

Las vistas, donde se programan todas las salidas HTML que el usuario final va a ver y con las que va a interactuar.

El código Javascript y CSS, por lo tanto, forma parte de las vistas.

En las vistas estará el grueso del código de cualquier aplicación. Los *frameworks* más avanzados incluyen sistemas de **plantillas** para reutilizar fragmentos de vistas, así como lenguajes adicionales para simplificar la codificación de las vistas. Pero, si programamos en PHP clásico, tendremos que construir las vistas manualmente.

6. RESUMEN

Los controladores, donde se captura cada petición del usuario y se dirige el flujo de ejecución, invocando a los modelos y a las vistas en el orden adecuado.

En una aplicación pequeña, bastará con tener un controlador para todo. Cuando la aplicación crece, suele hacerse **un controlador por cada modelo**, es decir, un controlador por cada tabla maestra.

Los controladores estarán compuestos por una colección de métodos, uno para cada funcionalidad de la aplicación.

6. RESUMEN

El método que se ejecute en cada ocasión estará controlado por la URL. En concreto, por la variable “*action*” que se pasará por GET, aunque, por supuesto, puedes ponerle otro nombre si “*action*” no te gusta.

En los *frameworks*, esta variable “*action*” se transforma en una URL limpia que, a través de un objeto adicional, llamado **enrutador**, termina provocando la invocación del método adecuado.

En una aplicación MVC tendremos una ruta como ésta:

```
https://mi-servidor/index.php?action=showArticle&idArticle=37
```



6. RESUMEN

En cambio, en un *framework* avanzado, la ruta tendrá un aspecto más limpio para hacer lo mismo. Algo así como:

```
https://mi-servidor/articles/show/37
```

Y el enrutador se encargará de trocear esa URL y extraer de ella la información para instanciar el controlador adecuado y llamar al método correcto.



6. RESUMEN

TAREA:

Transforma los ejercicios 1, 2, 5 y 6 del tema BBDD a una arquitectura MVC como la que hemos visto incluyendo en el modelo genérico las diferentes consultas que se pueden realizar.